Basic (plug search in as a plain tool first) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `search_tool_integration_practice.py`: this wraps a document search as a real `@tool` (Doc06's pattern) that the model decides whether to call — the exact wrapper the Build Task's search step reuses. A tiny in-memory `retrieve()` stands in for Doc08's vector search, so the file runs on its own without a real Chroma store. **If not:** the Build Task would be the first place you ever wrapped search as a `@tool`, with no smaller version to trust.

Solution

```
# search_tool_integration_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

MAX_RESULTS = 2

# A tiny stand-in for Doc08's knowledge base and retrieve(), inline so this
# file runs on its own. retrieve() returns a list of dicts, each with a
# "text" and its "source_file".
DOCUMENTS = [
    {"text": "A coarse grind suits a French press; a fine grind suits "
             "espresso.", "source_file": "grind.md"},
    {"text": "Store beans in an airtight container, away from light and "
             "heat.", "source_file": "storage.md"},
    {"text": "Use water just off the boil, around 93 C, for pour-over.",
     "source_file": "water.md"},
]
collection = DOCUMENTS


def retrieve(collection: list, query: str, k: int = MAX_RESULTS) -> list:
    # naive keyword overlap stands in for Doc08's vector search
    hits = []
    for document in collection:
        overlap = 0
        for word in query.lower().split():
```

```
            if word in document["text"].lower():
                overlap = overlap + 1
        if overlap > 0:
            hits.append(document)
    return hits[:k]


@tool
def search_docs(query: str) -> str:
    """Search the company's coffee documents for text relevant to the query.

    Use this whenever the question could be answered by looking something
    up in the coffee documents: brewing, grind size, water, storage.
    """
    try:
        results = retrieve(collection, query, k=MAX_RESULTS)
    except Exception as exc:
        return f"Search is temporarily unavailable: {exc}"
    if not results:
        return "No relevant documents found."
    pieces = []
    for result in results:
        pieces.append(result["text"])
    return "\n\n".join(pieces)


def answer(question: str) -> str:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    tools = [search_docs]
    messages = [{"role": "user", "content": question}]

    # think: let the model decide whether to call the search tool
    response = model.bind_tools(tools).invoke(messages)
    messages.append(response)

    # act: run every tool call and send the result back
    for call in response.tool_calls:
        result = search_docs.invoke(call["args"])
        messages.append({
            "role": "tool",
            "content": result,
            "tool_call_id": call["id"],
        })

    # think again, now with the search result in hand
    final = model.bind_tools(tools).invoke(messages)
    return final.content
```

```
if __name__ == "__main__":
    print(answer("What grind should I use for espresso?"))
```

Expected output (wording varies):

```
For espresso, use a fine grind.
```